

Each service was defined in a separate Dockerfile, specifying its runtime environment, dependencies, and execution commands. These containers were built as images and managed using Docker Compose during development for multi-container orchestration. Figure 5 displays the Docker Hub interface showing two public repositories: one for the Android frontend (price-scanner-android) and another for the scraping engine (price-scanner-scrapers). These images are automatically built and pushed via the CI/CD pipeline and later deployed to the Kubernetes cluster.

For deployment, the containerised services were orchestrated using a Kubernetes cluster. Kubernetes, being cloud provider-agnostic, offers capabilities like load balancing, horizontal scaling, self-healing, and service discovery. Pods were configured with resource limits and health checks to ensure optimal operation. Monitoring and debugging were carried out using the Kubernetes Dashboard, 'kubectl describe' and 'kubectl logs' commands, allowing real-time visibility into container behaviour and resource usage.

Monitoring and logging:

FastAPI logging middleware and Docker stdout streams were used for runtime logging. Each scraper logged execution time, item count, and errors. Backend logs included HTTP status codes and timestamps. Kubernetes tools provided insights into pod states, CPU/memory usage, and deployment health (Figure 6).

Ethical compliance: All scraping operations respected the *robots.txt* directives of the targeted platforms. Only publicly accessible pages were accessed, and no authentication-protected or rate-limited APIs were used. The data collection was conducted strictly for academic research, ensuring compliance with ethical and legal standards.

This comprehensive experimental setup validated the performance, reliability, and modularity of

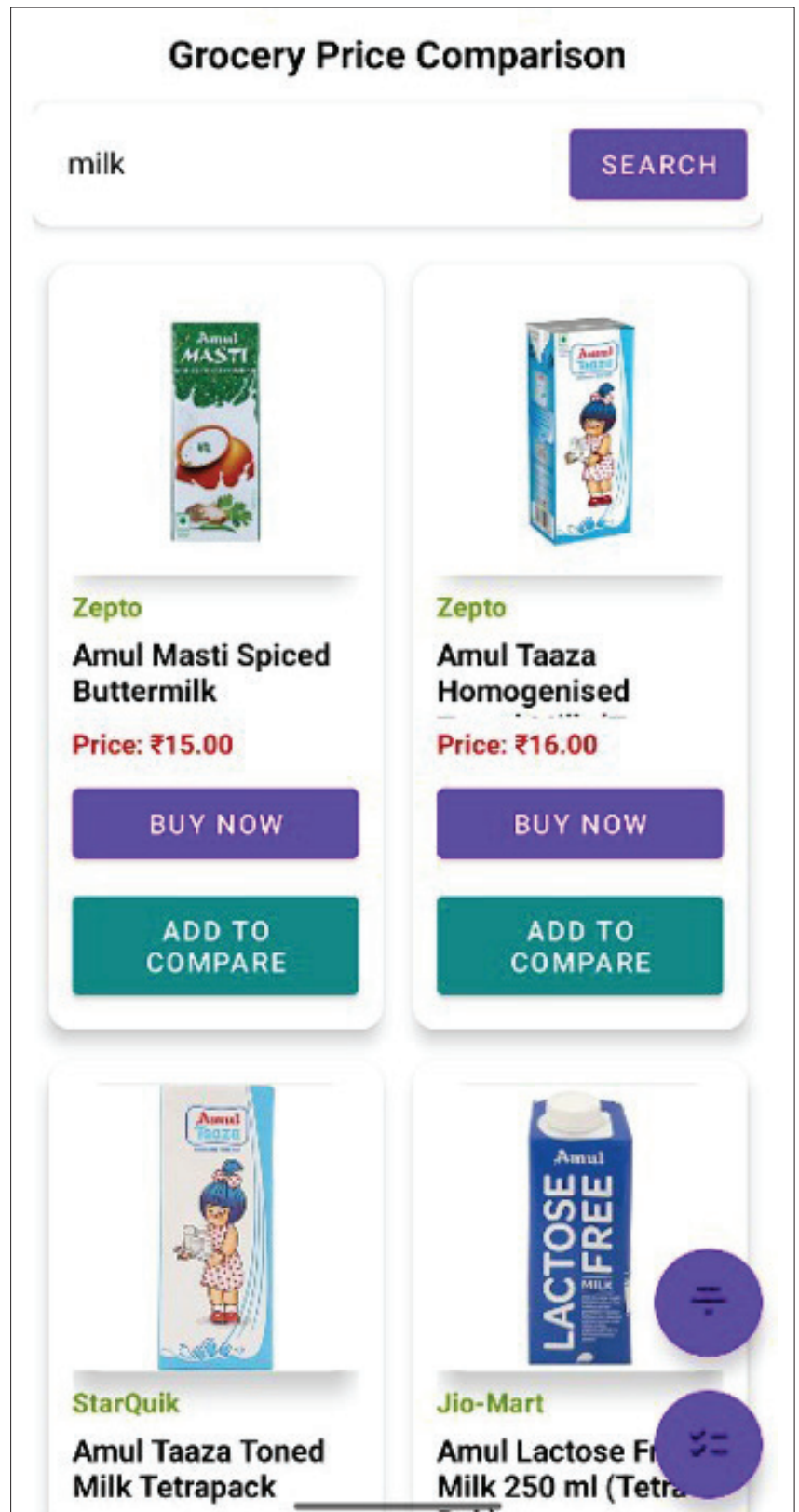


Figure 3: User interface for product search functionality with search bar and product listings